Bluestock Mutual Fund Analytics

End-to-End Data Pipeline & Risk Modeling Report

Prepared by: Dheeraj

July 2026

Table of Contents

1. Executive Summary
2. Project Objectives & Scope
3. Data Architecture & Data Dictionary
4. ETL Pipeline & Data Integrity
5. Exploratory Data Analysis: Macro Trends
6. Exploratory Data Analysis: Investor Demographics
7. Exploratory Data Analysis: Sector Allocations
8. Quantitative Performance Metrics (The Scorecard)
9. Advanced Risk Analytics: VaR & Drawdowns
10. Portfolio Diversification: Correlation Matrix
11. Business Intelligence: Executive Dashboard
12. Business Intelligence: Deep-Dive Analytics
13. Strategic Recommendations
14. Limitations & Future Scope
Appendix A: SQLite Schema Definition

Appendix B: Core Python Algorithms

1. Executive Summary

The Indian mutual fund industry has experienced unprecedented growth, surpassing ₹81 Lakh Crore in Assets Under Management (AUM). Despite this massive capital influx, retail investors often lack institutional-grade quantitative tools to assess risk-adjusted returns effectively. The Bluestock Mutual Fund Analytics project was conceptualized to solve this fragmentation by building a robust, automated end-to-end data pipeline.

This report details the successful extraction, transformation, and analysis of over 46,000 daily Net Asset Value (NAV) records for 40 top-performing funds. By engineering a unified SQLite data warehouse and deploying dynamic business intelligence dashboards, this project transitions raw API data into actionable financial intelligence.

Key deliverables include a proprietary composite scorecard system, advanced downside risk modeling (Value at Risk), and interactive data visualizations that highlight crucial demographic shifts toward systematic investing. The findings challenge the conventional reliance on simple Compound Annual Growth Rate (CAGR) by proving that risk-adjusted metrics (Sharpe, Alpha) provide a significantly more resilient framework for long-term wealth generation.

2. Project Objectives & Scope

2.1 Problem Statement

Currently, mutual fund data in India is highly fragmented. Investors are forced to rely on static PDF fact sheets or disparate web portals that do not allow for cross-sectional analysis. There is a distinct lack of open-source, automated reporting that unifies historical performance, expense ratios, and advanced risk metrics into a single source of truth.

2.2 Core Objectives

- **Automated Ingestion:** Build a Python-based ETL (Extract, Transform, Load) pipeline to autonomously fetch historical NAV data from the `mfapi.in` REST API.
- **Centralized Warehousing:** Design a relational SQLite database using a Star Schema to store and query millions of rows of financial data efficiently.
- **Quantitative Rigor:** Move beyond basic returns by programming advanced financial models, including 90-day rolling Sharpe ratios, Jensen's Alpha, and 95% Historical Value at Risk (VaR).
- **Interactive Visualization:** Develop a multi-page Business Intelligence dashboard (Power BI / Streamlit) equipped with dynamic slicers for real-time executive decision-making.

3. Data Architecture & Data Dictionary

3.1 Data Sources

The project relies on two primary data vectors. The first is live API data pulled from `mfapi.in`, which provides daily NAV updates for thousands of registered schemes. The second is curated metadata sourced from the Association of Mutual Funds in India (AMFI) registry, which provides static facts such as AMC names, fund categories, and benchmark indices.

3.2 Data Dictionary

The following table outlines the core facts table (`fact_nav_history`) established within the SQLite data warehouse:

Column Name	Data Type	Description
`date_key`	INTEGER	Primary Key (YYYYMMDD format) for time-series joining.
`scheme_code`	INTEGER	Foreign Key linking to AMFI Master scheme registry.
`nav_value`	REAL	The Net Asset Value of the fund on the given trading day.

`daily_return_pct`	REAL	Calculated day-over-day percentage change (used for VaR).
`is_holiday`	BOOLEAN	Flag indicating if the date was a market holiday/weekend.

4. ETL Pipeline & Data Integrity

4.1 The Forward-Fill (ffill) Imperative

A critical challenge in quantitative finance is handling non-trading days. Mutual funds do not update NAVs on weekends or public holidays. If these gaps are ignored or dropped, the time-series array collapses, artificially skewing volatility metrics and rendering rolling averages mathematically invalid.

To ensure data integrity, the pipeline implements a rigorous chronological reindexing protocol. Every fund's timeline is expanded to include every single calendar day between its inception and the current date. Pandas' `.ffill()` (forward-fill) method is then applied. This carries the Friday closing NAV through Saturday and Sunday, maintaining price continuity without fabricating market movements.

4.2 Outlier Detection

API sources are occasionally subject to data entry errors (e.g., a decimal misplaced in an NAV reading). The transformation layer of the ETL pipeline includes a Z-score outlier detection function. Any daily NAV movement exceeding a 4-standard-deviation threshold ($Z > 4$) triggers an automated quarantine alert, preventing corrupted data from entering the SQLite warehouse.

5. Exploratory Data Analysis: Macro Trends

Before analyzing individual funds, a macroscopic view of the Indian Mutual Fund industry was established to contextualize the current market environment. The standout metric is the explosive growth of Systematic Investment Plans (SIPs).

5.1 The SIP Surge

As illustrated in the chart below, SIP inflows reached an all-time high of ₹31,002 Crore in December 2025. This consistent monthly capital injection acts as a massive stabilizing force for the Indian equity markets, providing domestic institutional investors with reliable liquidity to absorb foreign outflows during global macro shocks.

[Paste Image: 03_sip_inflows.png Here]

Figure 1: Monthly SIP Inflows demonstrating exponential growth and market resilience.

6. Exploratory Data Analysis: Investor Demographics

Understanding *who* is driving the AUM growth is as important as measuring the growth itself. The demographic analysis uncovered a massive structural shift in how retail capital enters the market.

6.1 The Millennial Shift

Historically, mutual funds were utilized primarily by investors aged 45 and above as retirement vehicles. The data now indicates that the 26–35 age bracket forms the massive core of the investor base at 40.7%.

[Paste Image: 06_age_distribution.png Here]

Figure 2: Investor Age Group Distribution highlighting the 40.7% millennial dominance.

When combined with the 36-45 cohort, over 65% of the market is driven by younger demographics. This highlights a strong cultural shift toward systematic, tech-forward investing via mobile platforms rather than traditional banking channels.

7. Exploratory Data Analysis: Sector Allocations

A deep dive into the underlying portfolios of the Top 40 funds reveals distinct biases in sector allocation. Risk management requires understanding where these funds are deploying capital.

7.1 Financial Services Dominance

The analysis shows a heavy concentration in Banking and Financial Services across almost all large-cap and flexi-cap categories. While this sector drives Nifty earnings, it introduces high correlation risk. If the banking sector faces a liquidity crisis, a large percentage of purportedly "diversified" equity funds will suffer simultaneous drawdowns.

[Paste Image: 10_sector_donut.png Here]

Figure 3: Sector Allocation breakdown of the Top 5 Equity Funds.

8. Quantitative Performance Metrics

To rank the 40 mutual funds objectively, a Composite Fund Scorecard (0–100) was developed, replacing arbitrary CAGR-only comparisons with rigorous risk-adjusted mathematics.

8.1 The Scoring Algorithm

The Python pipeline ranked funds based on the following weighted pillars:

- **30% Returns (CAGR):** The Compound Annual Growth Rate over a 3-year horizon. We strictly utilize 252 trading days for annualization, avoiding the common mistake of using 365 calendar days.
- **25% Sharpe Ratio:** Calculates excess return per unit of total risk. A 6% annualized risk-free rate (divided by 252 for daily adjustment) was used as the baseline.
- **20% Jensen's Alpha:** Measures the manager's ability to beat the Nifty 100 benchmark. Positive alpha indicates true skill over mere market exposure.
- **15% Expense Ratio Penalty:** Funds charging higher Total Expense Ratios (TER) are penalized, as high fees severely compound against the investor over a 10-year horizon.
- **10% Maximum Drawdown:** The peak-to-trough decline during the worst market correction in the observed window.

9. Advanced Risk Analytics: VaR & Drawdowns

Institutional analysis prioritizes capital preservation. We utilized advanced statistical methods to quantify downside risk.

9.1 95% Historical Value at Risk (VaR)

VaR answers a simple question: "Under normal market conditions, what is my worst expected daily loss?" By isolating the 5th percentile of the daily returns array using Pandas, we identified the threshold that losses will not exceed 95% of the time.

9.2 Dynamic Rolling Sharpe

A single Sharpe ratio number hides volatility. The script calculated a 90-day rolling Sharpe ratio to visualize exactly how a fund's risk-adjusted performance reacted during specific macroeconomic shocks.

[Paste Image: rolling_sharpe_chart.png Here]

Figure 4: 90-Day Rolling Sharpe Ratio demonstrating dynamic volatility.

10. Portfolio Diversification: Correlation Matrix

Many retail investors believe they are diversified simply because they own 5 different mutual funds. However, if those 5 funds buy the exact same underlying stocks, the investor is exposed to systemic collapse.

10.1 Daily NAV Return Correlation

To prove true diversification, a pairwise correlation matrix was generated across the daily returns of the top-performing funds. Values approaching 1.0 indicate perfect correlation (they move together), while values near 0 indicate independence.

[Paste Image: 09_correlation_matrix.png Here]

Figure 5: Heatmap indicating the systemic overlap between top-performing mutual funds.

11. Business Intelligence: Executive Dashboard

Data science is only valuable if stakeholders can interact with the findings. An end-to-end interactive dashboard was deployed to synthesize the SQLite warehouse data.

11.1 The Macro View

The Executive Overview page was designed for high-level decision-makers. It aggregates massive datasets into digestible KPI cards, including Total AUM tracked, overall average expense ratios, and live SIP trajectory charts. Functional slicers allow the user to isolate data by Asset Management Company (AMC) or specific risk profiles instantly.

[Paste Screenshot of your Main Dashboard Page Here]

Figure 6: Executive Dashboard featuring interactive slicers and KPI metrics.

12. Business Intelligence: Deep-Dive Analytics

While executives require summaries, quantitative analysts require granularity. The secondary dashboard tab facilitates deep-dive inspections.

12.1 Fund Drill-Through

This page leverages a dynamic dropdown menu allowing the user to select any specific scheme from the 40 analyzed. Upon selection, the dashboard automatically recalculates the scheme's unique VaR, Maximum Drawdown, and overlays its historical NAV trajectory directly against the Nifty benchmark.

[Paste Screenshot of your Deep-Dive/Fund Comparison Dashboard Here]

Figure 7: Deep-Dive Analytics interface isolating specific scheme performance.

13. Strategic Recommendations

Based on the quantitative analysis and EDA findings, the following strategic recommendations are proposed for portfolio managers and retail platforms:

- **Prioritize Downside Resilience:** The performance modeling definitively shows that high-Sharpe, low-drawdown funds generate greater compound wealth over a 3-year period than high-CAGR funds that suffer massive corrections. Portfolios should be anchored by stability, not peak returns.
- **Implement Automated HHI Monitoring:** The Herfindahl-Hirschman Index (HHI) for sector concentration should be automated into client alerts. If a client's portfolio exceeds a 40% concentration in Financials or IT, the system should flag it as an over-exposure risk.
- **Deploy the Recommender Engine:** The logic built for the Python recommender script should be operationalized as an API. Retail brokerages can use this logic to match onboarding investors with mathematically appropriate funds based on an initial risk-tolerance questionnaire.

14. Limitations & Future Scope

14.1 Technical Constraints

The current pipeline successfully executes all ETL and mathematical operations; however, it is currently limited by local compute power and the API rate limits of `mfapi.in`. Bulk historical extraction can trigger throttling, requiring artificial sleep delays in the Python script.

14.2 Future Enhancements (Bonus Objectives)

The next iteration of this analytics suite will transition from local analysis to a fully automated cloud architecture:

- **Cron Job Automation:** Deploying the `etl_pipeline.py` script to an AWS EC2 instance or Lambda function to execute automatically every weekday at 8:00 PM IST to fetch daily closing NAVs.
- **Monte Carlo Forecasting:** Integrating stochastic simulations into the dashboard to project potential portfolio values 5 to 10 years into the future with 95% confidence bands.
- **Markowitz Optimization:** Enhancing the recommender system with the Efficient Frontier algorithm to output exact capital weightings (e.g., 30% to Fund A, 70% to Fund B) rather than just a list of funds.

Appendix A: SQLite Schema Definition

The foundational database architecture ensuring data integrity and rapid query performance:

```
-- AMFI Master Registry
CREATE TABLE dim_scheme (
    scheme_code INTEGER PRIMARY KEY,
    scheme_name TEXT NOT NULL,
    amc_name TEXT,
    category TEXT,
    sub_category TEXT,
    launch_date DATE
);

-- Daily NAV Fact Table
CREATE TABLE fact_nav (
    date_key INTEGER,
    scheme_code INTEGER,
    nav REAL,
    is_holiday BOOLEAN DEFAULT 0,
    FOREIGN KEY(scheme_code) REFERENCES dim_scheme(scheme_code),
    PRIMARY KEY (date_key, scheme_code)
);

-- Performance Scorecard
CREATE TABLE fact_metrics (
    scheme_code INTEGER PRIMARY KEY,
    cagr_3yr REAL,
    sharpe_ratio REAL,
    alpha REAL,
    var_95 REAL,
```

```
    final_score REAL
);
```

Appendix B: Core Python Algorithms

The proprietary mathematical logic used to generate the 90-Day Rolling Sharpe ratio, demonstrating the translation of financial theory into programmatic functions.

```
import pandas as pd
import numpy as np

def calculate_rolling_sharpe(returns_series, window=90,
risk_free_rate=0.06):
    # Computes the annualized rolling Sharpe Ratio.
    # Assumes standard 252 trading days.

    # Convert annual risk-free rate to daily
    daily_rf = risk_free_rate / 252

    # Calculate excess returns
    excess_returns = returns_series - daily_rf

    # Rolling mean of excess returns
    rolling_mean = excess_returns.rolling(window=window).mean()

    # Rolling standard deviation (Volatility)
    rolling_std = returns_series.rolling(window=window).std()

    # Annualize the result
    rolling_sharpe = (rolling_mean / rolling_std) * np.sqrt(252)

    return rolling_sharpe

# Applying to the cleaned, forward-filled DataFrame
df['rolling_sharpe'] = df.groupby('scheme_code')
['daily_return'].apply(
```

```
    lambda x: calculate_rolling_sharpe(x)
)
```